

시스템 안정성을 고려한 방어력 감소 메커니즘 설계

확장성, 공식 설계, 유저 친화적

시스템 설계

포트폴리오

목차

01

문제 정의

03

수치 검증 및 시뮬레이션

05

비판적 분석 및 개선안

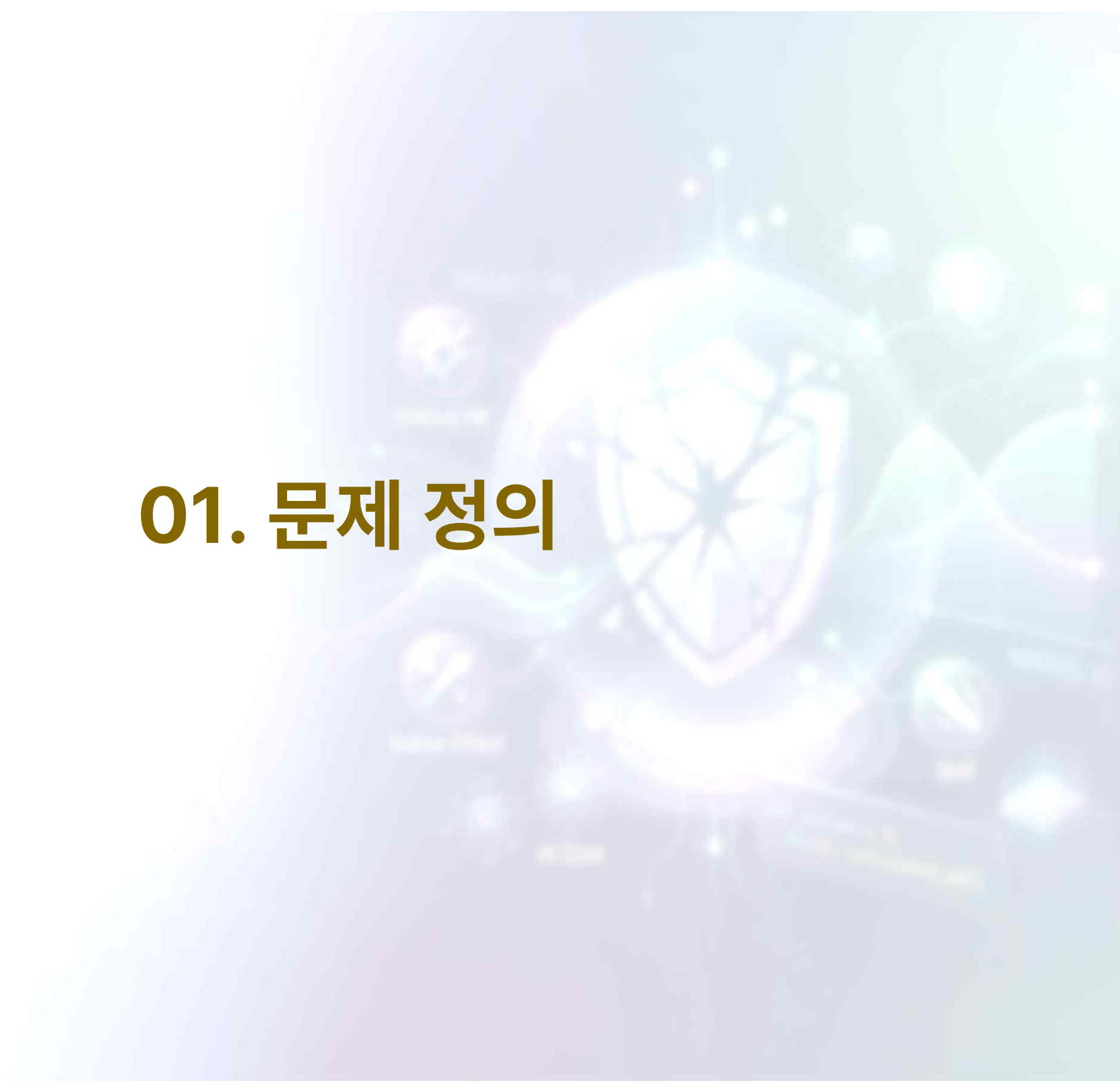
02

핵심 설계 논리

04

UX & 피드백 루프

01. 문제 정의



과제 배경

다양한 소스(아이템, 스킬, 패시브)에서 오는 방어력 감소 효과가 중첩될 때, 발생할 수 있는 시스템적 리스크를 고려한 설계 정의.

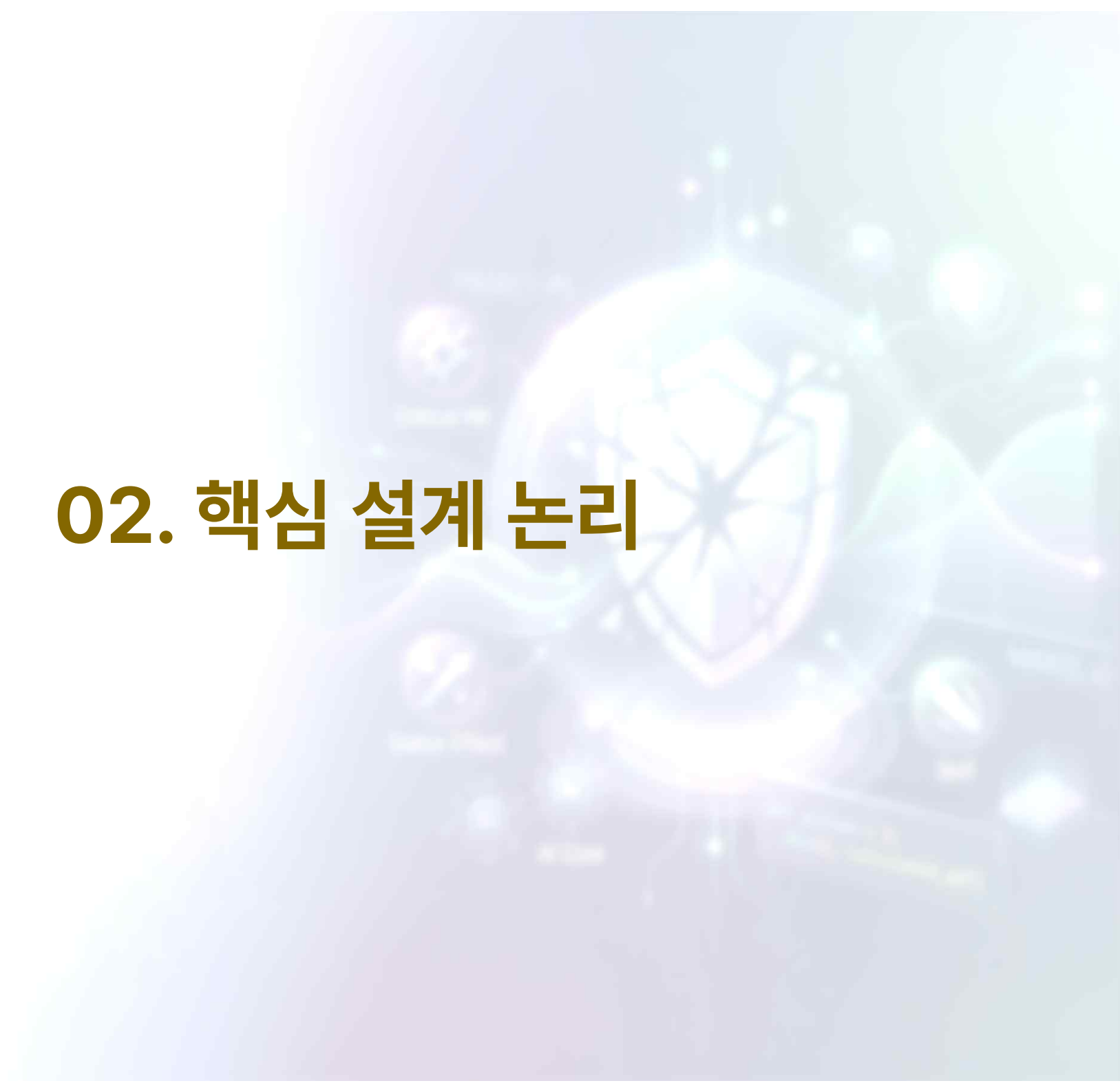
1. 수치 발산
 - > 방어력이 0이하로 떨어질 경우 발생하는 데미지 무한 발산
2. 직관성 결여
 - > 복잡한 연산 과정으로 유저가 자신의 강함을 인지하지 못하는 문제
3. 성장 불균형
 - > 초반/후반부 몬스터의 방어력 차이에 따른 디버프 효율의 변동



[데미지 무한 발산의 대표적인 예시]

- 단순 수치 오류가 아닌 실제 게임 진행에도 영향을 준다면 그것은 치명적인 버그이다.

02. 핵심 설계 논리



가정 환경

1. 최종 데미지 계산 방식은 아래와 같다고 가정한다.

$$(\text{기본 데미지}) * 100 / [100 + (\text{방어력}) * \{1 - (\text{방어력감소비율})\} * \{1 - (\text{방어력관통비율})\}]$$
2. 디버프는 퍼센트형과 정수형이 존재한다.

연산 순서

1. 결론 : 계산은 **퍼센트형 값을 우선적으로 계산한다.**

2. 이유

- > 퍼센트 계산이 우선이 되면, 퍼센트형 수치는 초반 저효율 후반 고효율로 자리잡음.
- > 이는 방어력 수치가 높을수록 퍼센트 효율이 높다는 **다수 유저 경험에 기반한 기대경험을 충족시킴.**
- > 또한, 이 방식은 정수형 디버프가 퍼센트형 디버프 계산 이후 적용이 되므로, **정수형 디버프의 가치를 초반부터 후반까지 어느정도 일정하게 유지하는 역할도 가능.**
- > 추가적으로, 이러한 계산 방식은 초반에는 정수형 디버프 투자를, 후반에는 퍼센트형 디버프 투자가 유리함으로 유도하기 때문에, 게임 설계에 있어 고려해야 함.

*퍼센트 우선 계산	방어력 50(초반)적	방어력 200(후반)적
정수형(30)	30	30
퍼센트형(30%)	$50 * 0.3 = 15$	$200 * 0.3 = 60$

연산 방법

1. 퍼센트형 값이 다중으로 중첩될 경우, **곱연산 방식을 사용한다.**
 - > 퍼센트형의 합연산은 방어력 감소 100% 초과 시, 데미지 공식의 붕괴를 야기함.
 - > 퍼센트형의 곱연산은 0에 수렴하는 구조라서 무한 중첩 상황에서 시스템 안정과 수치 제어가 가능
 - > 이는 급격한 데미지 인플레이션을 사전에 방지, 라이브 서비스 운영에도 유리

[계산 예]

상황 : 방어력 100의 방어자, 공격자는 각각 20%, 50%, 10%, 5, 7의 방어력 감소를 가지고 있음.

(최종 방어력)

$$\begin{aligned} &= \{(\text{기본 방어력}) * (\text{방어력 감소 비율})\} - (\text{정수형 방어력}) \\ &= \{100 * (1 - 0.2) * (1 - 0.5) * (1 - 0.1)\} - (5+7) \\ &= \{100 * (0.8 * 0.5 * 0.9)\} - 12 \\ &= \{100 * 0.36\} - 12 \\ &= 36 - 12 \\ &= \mathbf{24} \end{aligned}$$

2. 예외처리 및 가용범위를 설정한다.

- > 제안하는 데미지 공식에서 방어력이 음수가 되면, 데미지는 비정상적으로 증폭함
- > 정수형을 후순위로 계산하므로, **방어력이 0미만으로 떨어지는 것을 방지하는 조치 필요**
 - (최종 방어력) = $\text{Max}(0, \text{연산된 방어력})$

03. 수치 검증 및 시뮬레이션



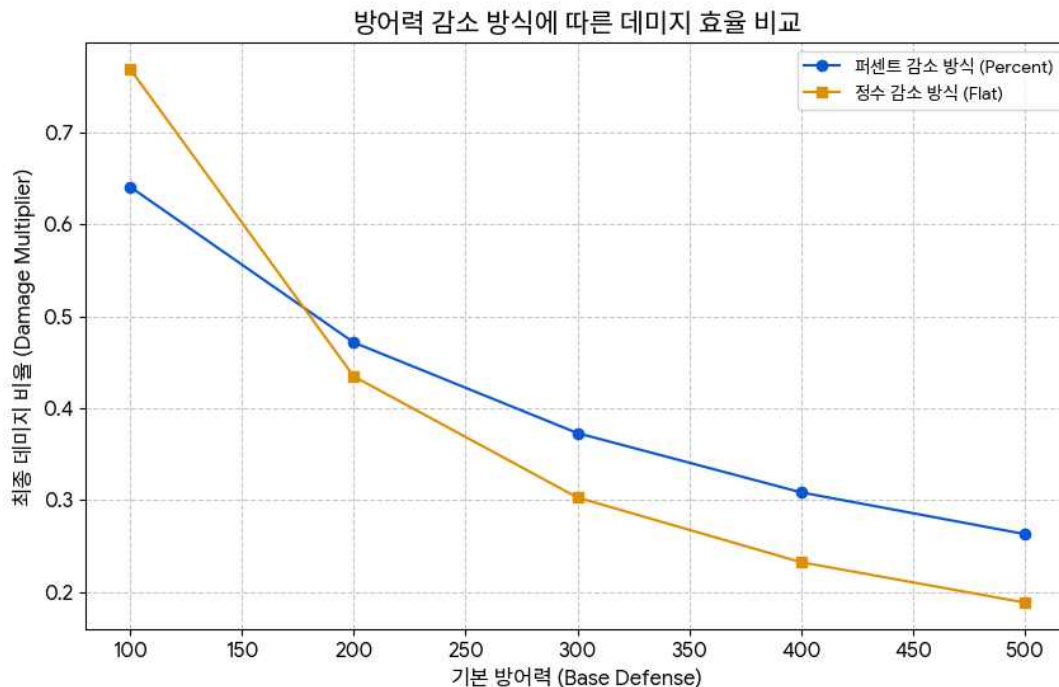
수치 검증 시뮬레이션

1. 방어력과 퍼센트 디버프, 정수형 디버프를 입력값으로 데미지 증폭 변화를 분석한다.

> 퍼센트형 디버프를 가진 A군과 정수형 디버프를 가진 B군.

> 제안한 공식이 적용된 시뮬레이션 구현 및 대입.

> 예측한대로, 낮은 방어력에서는 정수형이,
높은 방어력에서는 퍼센트형이 높은 효율성을 보임.



A군(퍼센트형)

- 30%
- 20%

B군(정수형)

- 50
- 20

시뮬레이션

- 제안한 연산방식 적용 후 최종 데미지율 계산
- 방어력 100, 200, 300, 400, 500의 경우

최종 데미지율(A군)

방어력	데미지율
100	0.641
200	0.4717
300	0.3731
400	0.3086
500	0.2632

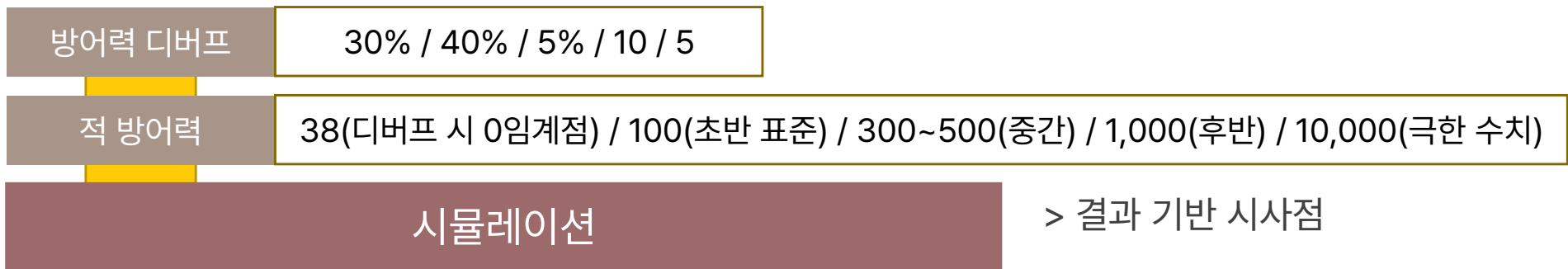
최종 데미지율(B군)

방어력	데미지율
100	0.7692
200	0.4348
300	0.303
400	0.2326
500	0.1887

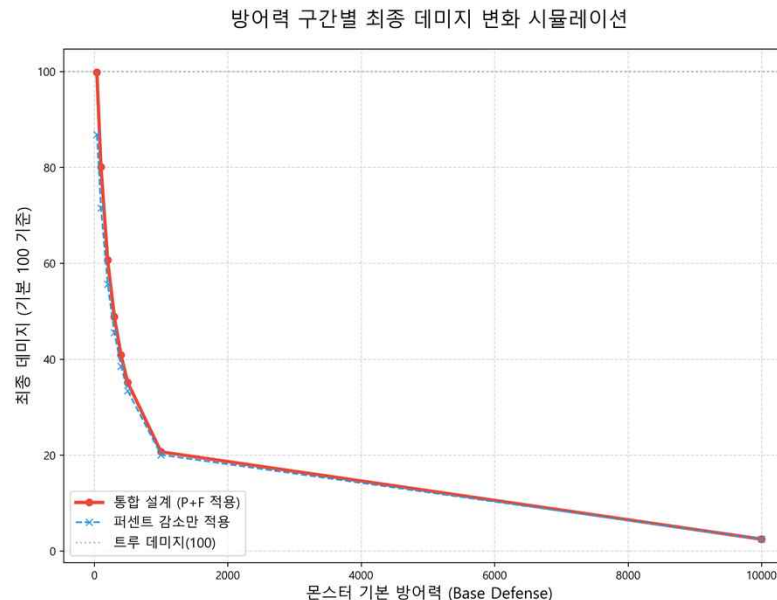
수치 검증 시뮬레이션

2. 해당 시뮬레이션으로 방어력 감소 특이점을 추적할 수 있다.

> 특정 수치의 두 디버프가 함께 적용될 때, 최적의 효율을 내는 방어력 추적.



최종 데미지율	
방어력	데미지율
38	0.998
100	0.8
300	0.489
400	0.409
500	0.352
1000	0.207
10000	0.025



> 결과 기반 시사점

- 시스템 안정성 : 방어력이 아무리 높더라도 일정한 데미지 비율이 유지가 되어, 특정 수치의 조합이 밸런스를 완전히 파괴하는 것을 방지
- 전략적 가치 : 방어력 40이하의 적에게는 정수 감소 수치가 결정적으로 작용, 방어력 0화가 더 잘게 일어나, 초반 유저의 성취감을 극대화.

결론 : 고방어력 적에게는 퍼센트 디버프가, 저방어력 적에게는 정수 디버프가 속도감을 결정짓는 구조를 가짐.

04. UX & 피드백 루프



복잡성의 추상화

- 유저가 강해진 것을 복잡한 숫자가 아닌 UX와 연출 피드백으로 전달한다.
 - > 유저에게 수식을 직접 노출 하는 것 대신, 시각적 계층을 통해 피드백을 전달.
 - > 굳이 계산하거나 공부하지 않아도 느껴지는 강함.

1. 방어력 감소율에 따른 방패파괴 아이콘의 단계별 변화(예시)
 - 타격 시, 감소된 적의 방어력 비율에 따른 연출 별도 표출



30% ~ 70%감소



70%~ 감소

복잡성의 추상화

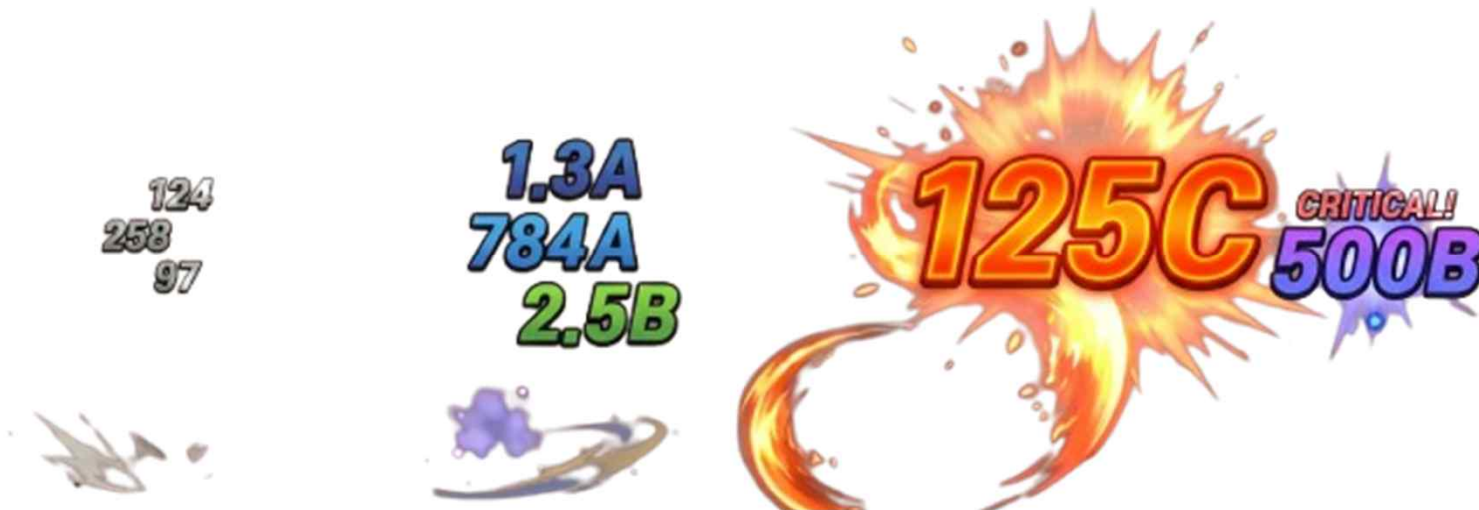
2. 치명타 적용 시 폰트 크기 및 연출 차별화

- 치명타 여부에 따른 별도 연출 추가
- 치명타 발생 시, 폰트 크기 증가 및 색상 채도 증가

일반적인 데미지 연출을 역으로 약하게 연출하여,
유저가 주로 경험하는 연출은 치명타가 효과가 적용된 연출이 되도록 유도합니다.
이는 치명타가 적용된 결과값으로 유저가 강해짐을 판단하게 유도하여,
일반 데미지로 느끼는 체감보다 더 높은 체감을 유발합니다.

3. 데미지 수치의 계층적 연출 차별화

- 1,000자리수마다 A~Z까지 차등 표출. (999 -> 1A -> 999A -> 1B....)
- 데미지 자리수 별로 색상이나 표출 방식/속도에 차이를 둬

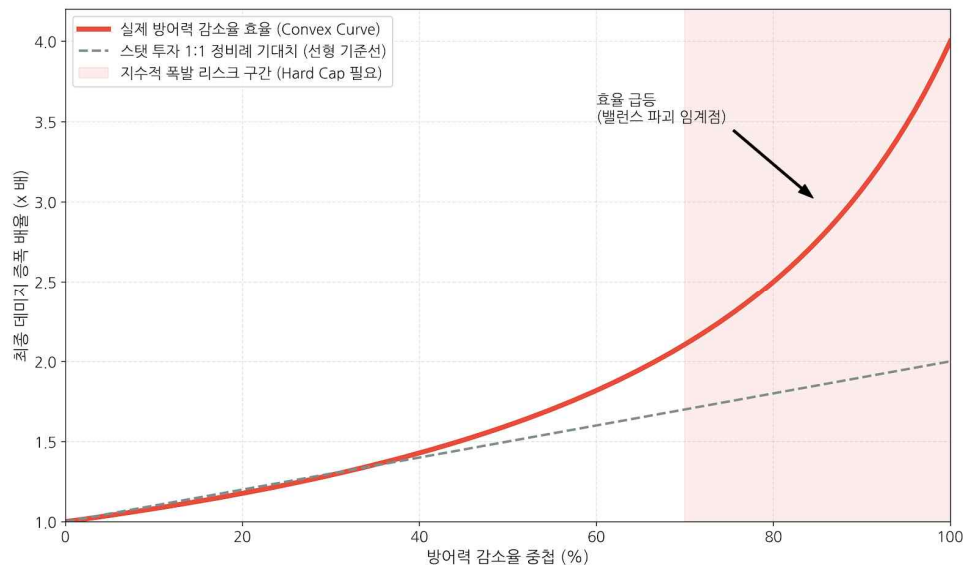


The background of the slide is a blurred, futuristic dashboard or control panel. It features various glowing blue and white elements, including circular gauges, buttons, and a central display area with a shield-like icon. The overall aesthetic is high-tech and digital.

05. 비판적 분석 및 개선안

자체 한계점 분석

- 방어력 감소 연산의 특정 지점에서의 데미지 급 증폭 리스크
 - > 단일 스탯의 밸런스 붕괴 리스크
 - 실제 시뮬레이션을 진행해보면, 현 구조는 직관에 반하는 현상이 존재함



- 직관에 기반한 방어력 감소율의 효율 : 점선
- 실제 방어력 감소율의 효율 : 붉은 실선
- 현 구조의 방어력 감소율은 특정값을 기점으로 폭발적으로 효율이 증가함
- 방어력 300, 기본공격력 100 기준

> 메타 고착화

- 해당 현상 방치 시, 파티 플레이 시, 유저들은 오직 방어력 감소 중첩을 100%로 만드는 것을 강제 받게 되어, 다른 스탯의 가치를 훼손하고 전략적 다양성을 파괴하게 됨

해결방안

- 연산 레이어 분리를 통한 분산 투자 유도
 - > 의도된 난이도 기반 최대 감소율 제한
 - 적에 따라 방어력 감소 디버프의 최대 중첩 한계선을 설정
 - 해당 내용은 유저에게 필히 명시(적 조우, 시스템적 연출 등)
 - > 데미지에 영향을 주는 수치를 신설, 독립 계산
 - 방어력 연산이 끝난 후 최종 데미지에 곱해지는 별도 레이어를 설계.
 - # 이를 통해 파티원들이 '방어력 감소'와 '피해 증가'로 역할을 분담하도록 유도하고, 각 유저가 투자한 스탯이 정직한 시너지 체감으로 이어지도록 전투 환경을 구조화합니다.
 - # 데미지 공식 변경 예시
$$(\text{기존 데미지 공식}) * \{1 + \text{데미지증가비율}\} * (1 - \text{피해보정비율})$$
 - > 비선형 요소 도입
 - 치명타, 회피율과 같은 비선형적 요소 도입
 - # 단, 비선형 요소는 도입 시 실효체력(EHP : Effective HP)에 대한 추가적 설계가 필요하므로, 게임의 방향성(캐주얼, 하드코어 RPG 등)에 따라 도입여부나 그 정도를 조절해야 합니다.



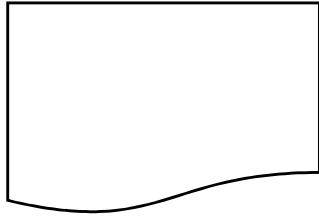
부록

시뮬레이션 코드

입력데이터

시뮬레이션(python3.13, vs code)

출력데이터



input_defense_data.csv

[Label1(필수)]
- 기본 방어력(int/float)

[Label P_Debuff_{n}]
- 퍼센트 디버프
- 0~1 float

[Label F_Debuff_{n}]
- 정수형 디버프
- int

```
import pandas as pd
import matplotlib.pyplot as plt
import platform
import os
from matplotlib import font_manager, rc

def set_korean_font():
    if platform.system() == 'Windows':
        rc('font', family='Malgun Gothic')
    elif platform.system() == 'Darwin':
        rc('font', family='AppleGothic')
    else:
        try: rc('font', family='NanumGothic')
        except: pass
    plt.rcParams['axes.unicode_minus'] = False
```

필요도구 import

```
def run_dynamic_simulation():
    set_korean_font()

    current_dir = os.path.dirname(os.path.abspath(__file__))
    input_path = os.path.join(current_dir, 'input_defense_data.csv')
```

```
    if not os.path.exists(input_path):
        print(f"X 에러: {input_path} 파일을 찾을 수 없습니다!")
        return
```

```
    df = pd.read_csv(input_path)
    base_dmg = 100

    p_cols = [c for c in df.columns if 'P_Debuff' in c]
    f_cols = [c for c in df.columns if 'F_Debuff' in c]

    print(f"🕒 인식된 퍼센트 디버프 항목: {p_cols}")
    print(f"🕒 인식된 정수 디버프 항목: {f_cols}")
```

입력데이터 로드

```
    def calculate_logic(row):
        base_def = row['Base_Defense']

        p_reduction_ratio = 1.0
        for col in p_cols:
            p_reduction_ratio *= (1 - row[col])

        f_reduction_total = sum(row[f_cols])

        def_only_p = base_def * p_reduction_ratio
        def_integrated = max(0, def_only_p - f_reduction_total)

        dmg_only_p = base_dmg * (100 / (100 + max(0, def_only_p)))
        dmg_integrated = base_dmg * (100 / (100 + def_integrated))

        return pd.Series([dmg_only_p, dmg_integrated])

    df[['Dmg_Only_P', 'Dmg_Integrated']] = df.apply(calculate_logic, axis=1)
```

데미지 계산

```
output_csv = os.path.join(current_dir, 'simulation_result_dynamic.csv')
df.to_csv(output_csv, index=False, encoding='utf-8-sig')
```

결과 저장 및 시각화

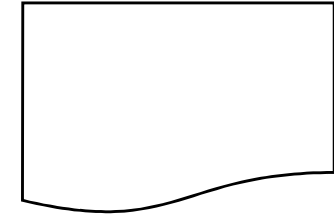
```
plt.figure(figsize=(11, 7))
plt.plot(df['Base_Defense'], df['Dmg_Integrated'], marker='o', lw=3, color='#e74c3c', label='통합 설계 (P+F 적용)')
plt.plot(df['Base_Defense'], df['Dmg_Only_P'], marker='x', ls='--', color='#3498db', label='비교군 (퍼센트만 적용)')

plt.title('데이터 기반 방어력 감소 효율 시뮬레이션', fontsize=18, pad=20)
plt.xlabel('몬스터 기본 방어력 (Base Defense)', fontsize=13)
plt.ylabel('최종 데미지 (기준 100)', fontsize=13)

plt.axhline(y=100, color='gray', ls=':', alpha=0.5, label='트루 데미지(100)')
plt.grid(True, alpha=0.3)
plt.legend(fontsize=11)

output_png = os.path.join(current_dir, 'dynamic_damage_graph.png')
plt.savefig(output_png, dpi=300)
print(f"📄 시뮬레이션 완료! \nCSV: {output_csv} \nPNG: {output_png}")
plt.show()
```

```
if __name__ == "__main__":
    run_dynamic_simulation()
```



simulation_result_dynamic.csv

[추가 Label1]
- Dmg_Only_P
- 퍼센트만 적용된 데미지율

[추가 Label2]
- Dmg_Integrated
- 방어력 디버프가 적용된 데미지율

[결과 시각화]

